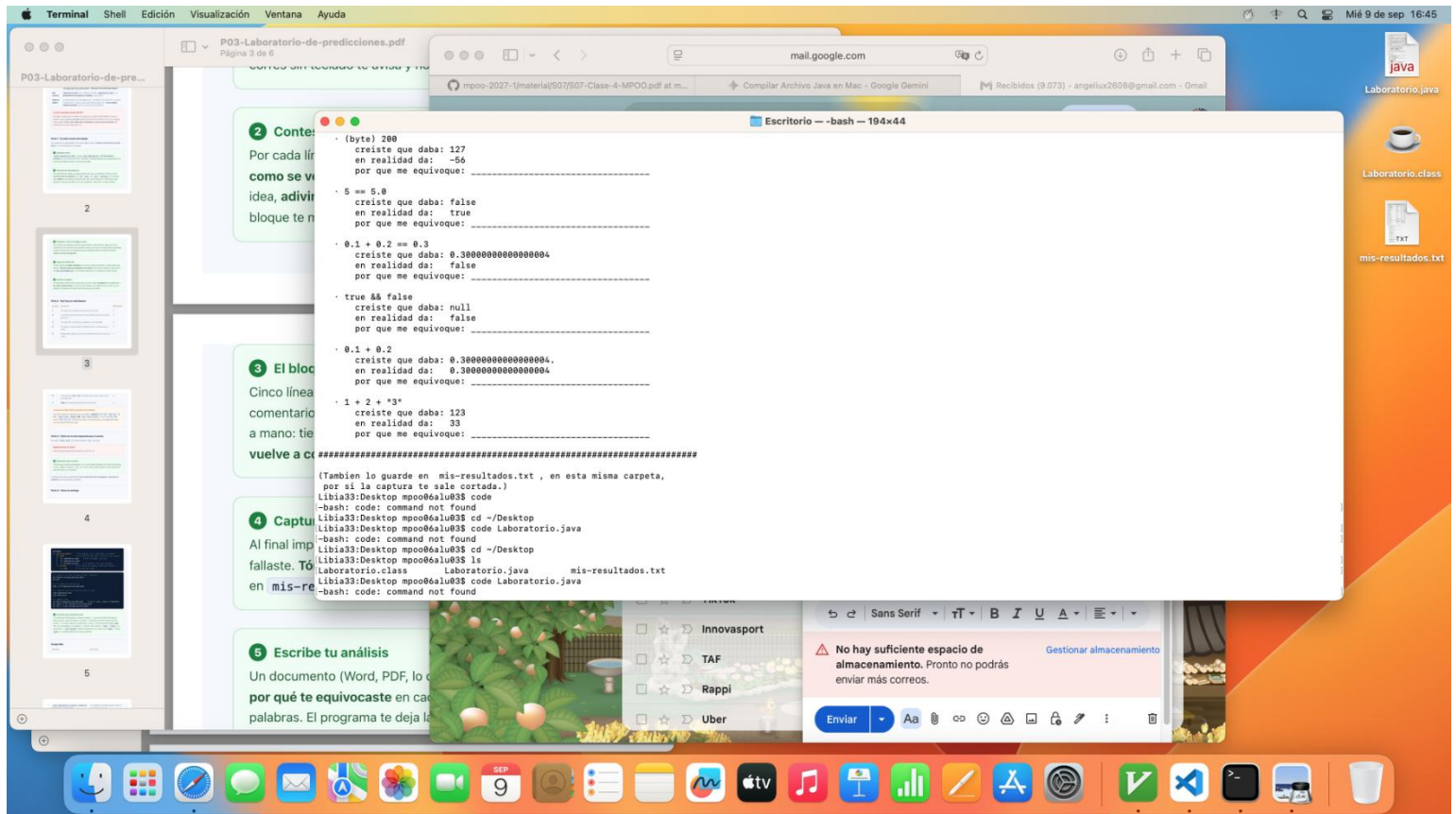




(double) (7 / 2)

creíste que daba: 3.5

en realidad, da: 3.0

por qué me equivoque: Pensé que el (double) convertía los números a decimales antes que se haga la operación, pero los paréntesis hacen que se resuelva primero la división entera (que da 3), y ya después ese resultado se convierte a decimal (3.0).

0.0 / 0.0

creíste que daba: Infinity

en realidad, da: NaN

por qué me equivoque: A pesar de que sí sabía que la división entre cero es una indeterminación matemática, pensé que al usar decimales ocurriría algún caso raro por lo que la otra opción era que daba infinito, pero fue errónea mi intuición.

(int) 'A'

creíste que daba: 0

en realidad, da: 65

por qué me equivoque: Pensé que convertir una letra a entero devolvería 0 por no tener valor numérico, pero en programación los caracteres se guardan mediante un código numérico interno (ASCII), después de responderlo recordé ese detalle, por lo que la A se asocia al número 65.

(byte) 200

creíste que daba: 127

en realidad, da: -56

por qué me equivoqué: Pensé que si un valor superaba el límite máximo de un byte (127), se quedaba estancado en ese tope, pero en realidad el tipo de dato como que da la vuelta hacia los números negativos, terminando en -56.

5 == 5.0

creíste que daba: false

en realidad, da: true

por qué me equivoque: Pensé que la comparación daría falso por tratarse de tipos distintos (entero y decimal), pero el lenguaje convierte automáticamente el entero a decimal antes de comparar, evaluando $5.0 == 5.0$ como verdadero.

0.1 + 0.2 == 0.3

creíste que daba: 0.30000000000000004

en realidad, da: false

por qué me equivoqué: Tuve un error conceptual, escribí el resultado número junto con la imprecisión que vimos en clase, pero el operador `==` es una comparación que nos debe devolver un verdadero o falso, en este caso falso.

true && false

creíste que daba: null

en realidad, da: false

por qué me equivoque: Pensé que al combinar verdadero y falso sin coincidir devolvería un valor vacío (null), pero los operadores lógicos `&&` siempre devuelven un valor booleano (true o false), dando falso si alguno de los dos no se cumple.

0.1 + 0.2

creíste que daba: 0.30000000000000004.

en realidad, da: 0.30000000000000004

por qué me equivoque: Cometí un error tipográfico, si había entendido el resultado con su imprecisión porque lo habíamos visto en clase, peor agregue un punto final por accidente.

1 + 2 + "3"

creíste que daba: 123

en realidad, da: 33

por qué me equivoque: Pensé que la presencia del texto "3" hacía que todos los elementos se unieran como texto desde el inicio, pero las operaciones se evalúan de izquierda a derecha: primero suma los números $1 + 2$ (3) y luego pega ese resultado con el texto "3".